

# PJ

## Task

- 用javafx实现这个lab系列的图形界面

## Coding Requirements

1. 之前lab中在字符界面上显示的内容，pj中改用图形界面显示。
  - a. 对于界面的布局的要求：左侧展示棋盘，右侧分3列展示其他信息（见运行示例）。其中第2列的游戏列表要求使用ListView形式。
  - b. 界面中应当包含之前lab中涉及的内容：带序号的棋盘、当前游戏编号、黑方和白方的名字及分数、当前轮到哪一方下棋、游戏列表、剩余炸弹数量、游戏结束时的结果展示等等。
  - c. （新要求）调整界面窗口大小时，棋盘部分需要固定位置，其余3个信息区域要均匀占据剩余的空间。
  - d. 原先在字符界面中显示的提示词，在pj图形界面中不做要求。（请玩家[Tom]输入落子位置(1A) / 炸弹位置(@1A) / 游戏编号(1-3) / 新游戏类型(peace/reversi/gomoku) / 演示模式(playback) / 退出程序(quit):)
  - e. 不同的游戏类型，对应图形界面有所不同，具体要求同之前lab。例如只有reversi需要显示双方分数，只有gomoku需要显示剩余炸弹数量等。
2. 之前lab中通过键盘输入来操控游戏，pj中改用鼠标点击相关组件的方式来操控游戏。
  - a. 下棋：键盘输入 1a --> 鼠标点击 1a 对应的棋盘格子。
  - b. 切换游戏：键盘输入 2 --> 鼠标点击 game 2 对应的按钮。
  - c. 新建游戏：键盘输入 `gomoku` --> 鼠标点击 新建gomoku 对应的按钮。
  - d. reversi的pass功能：键盘输入 pass --> 鼠标点击 pass 对应的按钮。
  - e. gomoku的炸弹功能：键盘输入 @1a --> 鼠标点击 炸弹 对应的按钮，再点击 1a 对应的棋盘格子。
  - f. 异常情形的报错：图形界面不做要求。可以无反应，可以弹窗，可以在界面的特定区域显示相关文字信息。（这是一种有效的调试方法）
  - g. 退出游戏：键盘输入 quit --> 鼠标点击 quit 对应的按钮。  
（新要求）退出游戏的同时需要做状态保存（见第5点）。

- h. 演示模式：键盘输入 `playback test1.cmd` --> 鼠标点击 `演示模式` 对应的按钮，选择 `test1.cmd` 文件。读入指令与lab6要求相同，演示效果是GUI界面上以固定的时间间隔显示每一步执行之后的状态。
- 3. 游戏规则同之前的lab，其中五子棋的规则以lab6为准（15\*15棋盘、障碍物、炸弹）
- 4. lab6中的演示模式，在pj中也需要实现。
- 5. （新要求）状态保存与恢复：游戏退出时，需要把游戏的当前状态保存到一个文件中(比如 `pj.game` )。启动游戏的时候，如果当前目录下有 `pj.game` 文件，能够恢复游戏当前的状态，恢复的内容包括：
  - a. 游戏列表
  - b. 每一盘游戏退出时候的状态
  - c. 当前运行的游戏（应当就是退出时正在运行的游戏）

## Document Requirements

- 1. Lab6是最后一个基于字符界面的版本，GUI版本应该多少会复用Lab6中的部分内容。最好的结果是，Lab6中与游戏逻辑相关的代码没有修改，直接共享，而有些设计则需要进行修改才能使用。请简单分析Lab6和Pj代码的复用情况。
- 2. 简单分析一下，如果需要加一个新的game类型，比如2048游戏，需要在上述结构上做哪些调整。要求使用图形给出代码的结构说明（类、模块和依赖关系）。

# 运行示例（仅供参考）

## 1. 初始界面

A B C D E F G H

1

2

3

4

5

6

7

8

游戏 1

黑棋 [Tom]

白棋 [Jerry]

演示模式

退出游戏

游戏列表

1.peace

2.reversi

3.gomoku

添加新游戏

添加Peace

添加Reversi

添加Gomoku

A B C D E F G H

1

2

3

4

5

6

7

8

游戏 2

黑棋 [Tom]: 2

白棋 [Jerry]: 2

跳过本轮

演示模式

退出游戏

游戏列表

1.peace

2.reversi

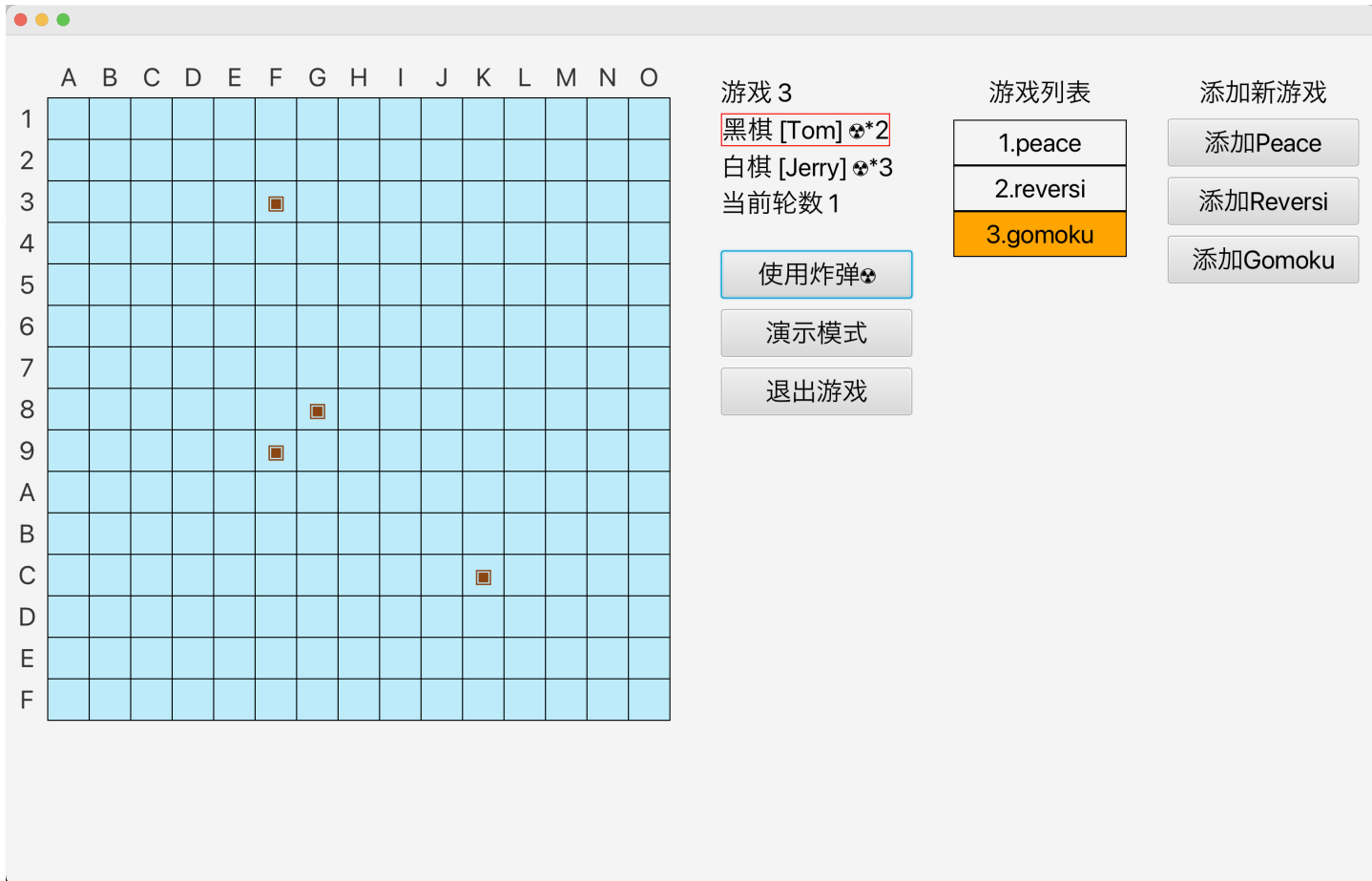
3.gomoku

添加新游戏

添加Peace

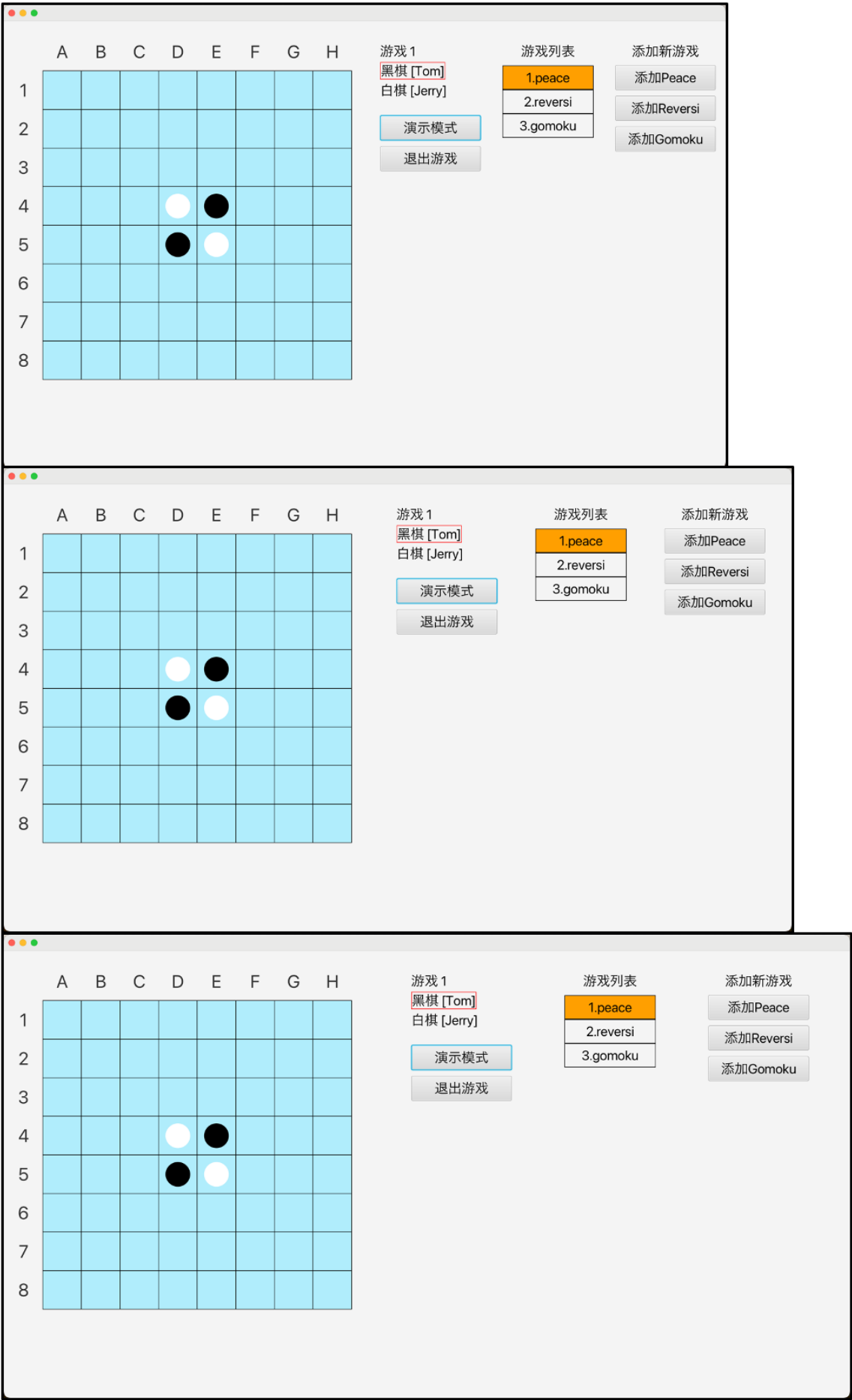
添加Reversi

添加Gomoku



2. (1.c) 调整窗口界面大小时的效果

- 宽度调整



高度调整

A B C D E F G H

1

2

3

4

5

6

7

8

游戏 1

黑棋 [Tom]

白棋 [Jerry]

演示模式

退出游戏

游戏列表

1.peace

2.reversi

3.gomoku

添加新游戏

添加Peace

添加Reversi

添加Gomoku

A B C D E F G H

1

2

3

4

5

6

7

8

游戏 1

黑棋 [Tom]

白棋 [Jerry]

演示模式

退出游戏

游戏列表

1.peace

2.reversi

3.gomoku

添加新游戏

添加Peace

添加Reversi

添加Gomoku

3. Gomoku放置炸弹

- 当前轮到白方落子

A B C D E F G H I J K L M N O

1

2

3

4

5

6

7

8

9

A

B

C

D

E

F

游戏 3

黑棋 [Tom] ♣\*2

白棋 [Jerry] ♣\*3

当前轮数 4

使用炸弹 ♣

演示模式

退出游戏

游戏列表

1.peace

2.reversi

3.gomoku

添加Peace

添加Reversi

添加Gomoku

添加新游戏

- 白方点击“使用炸弹”按钮

A B C D E F G H I J K L M N O

1

2

3

4

5

6

7

8

9

A

B

C

D

E

F

游戏 3

黑棋 [Tom] ♣\*2

白棋 [Jerry] ♣\*3

当前轮数 4

使用炸弹 ♣

演示模式

退出游戏

游戏列表

1.peace

2.reversi

3.gomoku

添加Peace

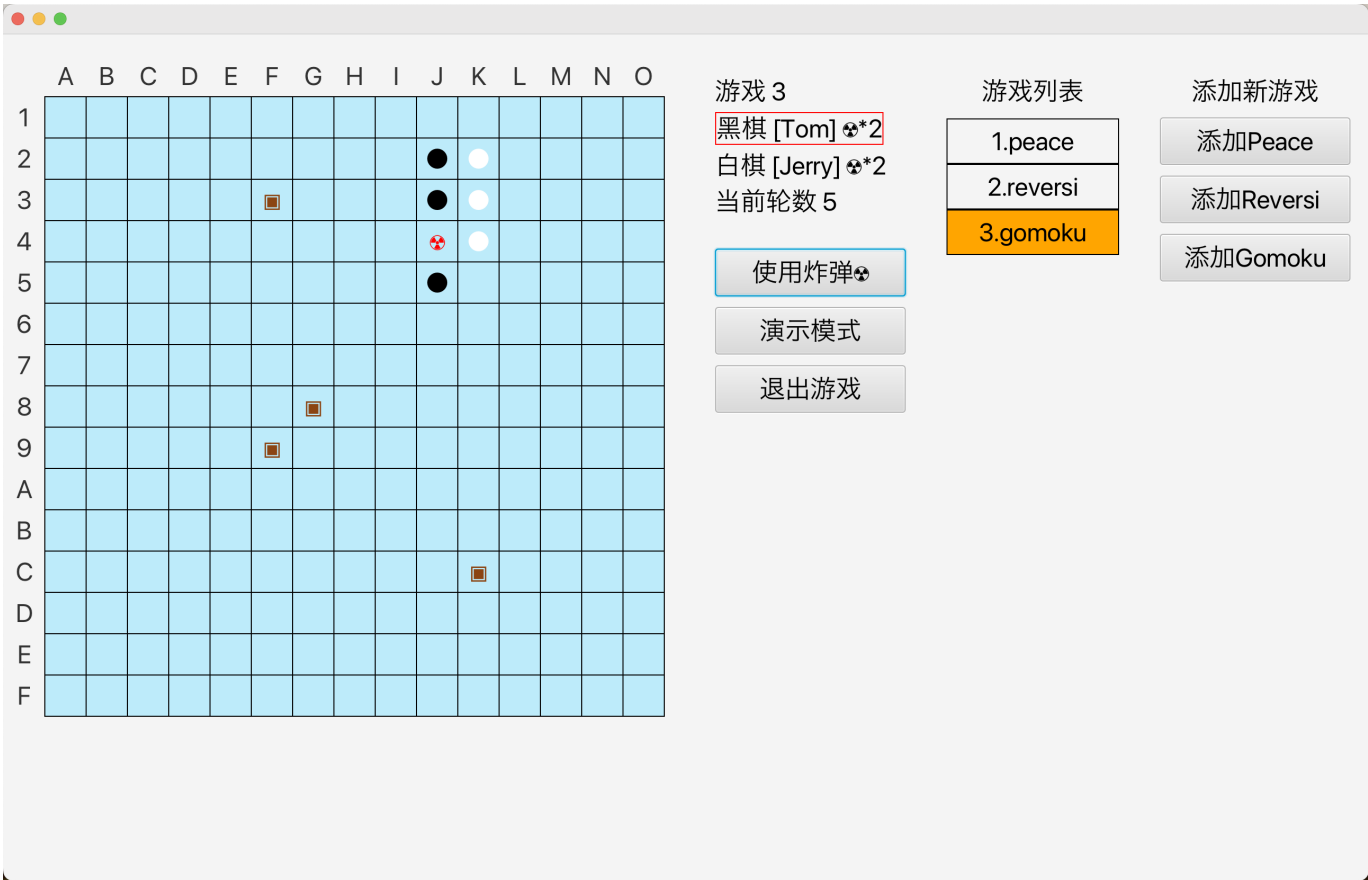
添加Reversi

添加Gomoku

添加新游戏

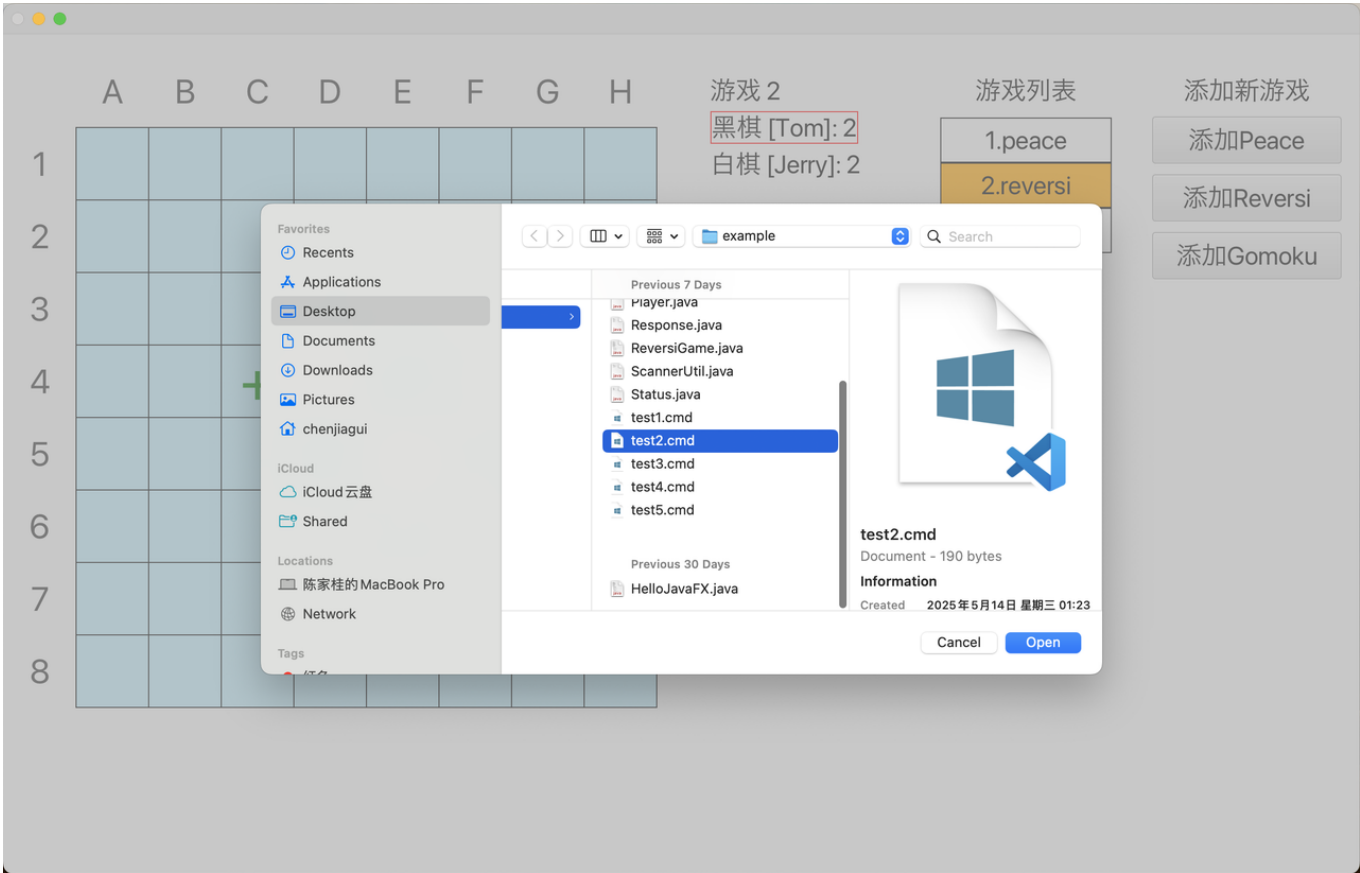
请放置炸弹！

- 白方点击4J放置炸弹



4. 演示模式（以Reversi为例）

- 点击“演示模式”按钮，选择对应的指令文件



- 每间隔一段时间执行一步指令并显示对应的GUI状态。最终结果如下：

A

B

C

D

E

F

G

H

1

2

3

4

5

6

7

8

|  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |

游戏 2

黑棋 [Tom]: 36

白棋 [Jerry]: 28

跳过本轮

演示模式

退出游戏

游戏列表

1.peace

2.reversi

3.gomoku

添加新游戏

添加Peace

添加Reversi

添加Gomoku

游戏结束！

黑棋 [Tom] 36

白棋 [Jerry] 28

Player1 [Tom] 获胜！

- 演示模式执行之后，应当可以继续下棋或者执行其他操作（例如新建游戏）

A

B

C

D

E

F

G

H

1

2

3

4

5

6

7

8

|  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |

游戏 2

黑棋 [Tom]: 36

白棋 [Jerry]: 28

跳过本轮

演示模式

退出游戏

游戏列表

1.peace

2.reversi

3.gomoku

4.peace

5.peace

6.peace

7.peace

8.peace

9.peace

添加新游戏

添加Peace

添加Reversi

添加Gomoku

游戏结束！

黑棋 [Tom] 36

白棋 [Jerry] 28

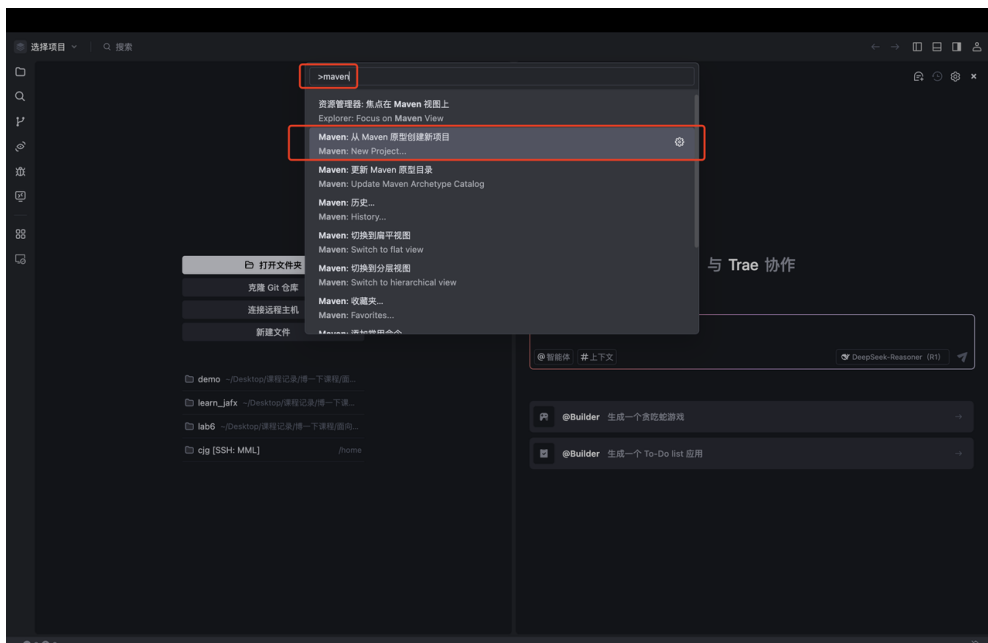
Player1 [Tom] 获胜！

# Submission

- 提交内容：一个压缩文件，以“学号\_姓名\_pj.zip”命名。包含项目代码和实验报告
  - 请将所有要提交文件放在学号\_姓名\_pj文件夹下再压缩，而不是直接压缩文件夹后改名
- 项目代码：包括所有Java源代码文件、编译后的可执行文件
- 实验报告：一个pdf文件，以“学号\_姓名\_pj.pdf”命名，包含必要的项目说明。包括但不限于：
  - 源代码文件名称及其主要功能
  - 关键代码及其设计思路
  - 运行过程截图及简单说明（选择只写增加代码思路也需要运行截图和过程说明）
- 截止时间：2025/06/19 23:59（期末考试后一天）

## 构建javafx项目

- 环境配置
  - 配置maven: <https://juejin.cn/post/7390413118889033768>
    - 验证安装: `mvn -version`
  - 配置javafx: [https://blog.51cto.com/u\\_16175524/12586021](https://blog.51cto.com/u_16175524/12586021)
- 在pom.xml中填写javafx相关的依赖、配置信息
- 打包&运行：执行 `mvn javafx:run`
- vscode新建maven项目



- pom.xml配置文件（参考）

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.example</groupId>
<artifactId>demo</artifactId>
<version>1.0.0</version>
```

```
<name>demo</name>
<!-- FIXME change it to the project's website -->
<url>http://www.example.com</url>
```

```
<properties>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
<maven.compiler.source>23</maven.compiler.source>
<maven.compiler.target>23</maven.compiler.target>
</properties>
```

```
<dependencies>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>4.11</version>
<scope>test</scope>
</dependency>
<dependency>
<groupId>org.openjfx</groupId>
<artifactId>javafx-controls</artifactId>
<version>21</version>
</dependency>
</dependencies>
```

```
<build>
```

```
<pluginManagement><!-- lock down plugins versions to avoid using Maven defaults (may be moved to parent pom) -->
```

```
<plugins>
```

```
<plugin>
```

```
<groupId>org.openjfx</groupId>
```

```
<artifactId>javafx-maven-plugin</artifactId>
```

```
<version>0.0.8</version>
```

```
<configuration>
```

```
<mainClass>com.example.HelloJavaFX</mainClass> <!-- 替换为你的主类 -->
```

```
</configuration>
```

```
</plugin>
```

```
<!-- clean lifecycle, see https://maven.apache.org/ref/current/maven-core/lifecycles.html#clean\_Lifecycle -->
```

```
<plugin>
```

```
<artifactId>maven-clean-plugin</artifactId>
```

```
<version>3.1.0</version>
```

```
</plugin>
```

```
<!-- default lifecycle, jar packaging: see https://maven.apache.org/ref/current/maven-core/default-bindings.html#Plugin\_bindings\_for\_jar\_packaging -->
```

```
<plugin>
```

```
<artifactId>maven-resources-plugin</artifactId>
```

```
<version>3.0.2</version>
```

```
</plugin>
```

```
<plugin>
```

```
<artifactId>maven-compiler-plugin</artifactId>
```

```
<version>3.8.0</version>
```

```
</plugin>
```

```
<plugin>
```

```
<artifactId>maven-surefire-plugin</artifactId>
```

```
<version>2.22.1</version>
```

```
</plugin>
```

```
<plugin>
```

```
<artifactId>maven-jar-plugin</artifactId>
```

```
<version>3.0.2</version>
```

```
</plugin>
```

```
<plugin>
```

```
<artifactId>maven-install-plugin</artifactId>
```

```
<version>2.5.2</version>
```

```
</plugin>
```

```
<plugin>
  <artifactId>maven-deploy-plugin</artifactId>
  <version>2.8.2</version>
</plugin>
<!-- site lifecycle, see https://maven.apache.org/ref/current/maven-
core/lifecycles.html#site_Lifecycle -->
<plugin>
  <artifactId>maven-site-plugin</artifactId>
  <version>3.7.1</version>
</plugin>
<plugin>
  <artifactId>maven-project-info-reports-plugin</artifactId>
  <version>3.0.0</version>
</plugin>
</plugins>
</pluginManagement>
</build>
</project>
```

- 可以在pom.xml中配置maven-jar-plugin + maven-dependency-plugin打jar包
  - mvn clean package
  - 该方法生成的jar包不包含依赖，需要下载对应版本的javafx sdk才能运行
    - <https://openjfx.cn/dl/>

## 评分标准

| 类型 | 项目        | 需求            |
|----|-----------|---------------|
| 代码 | peace模式   | 按照规则下棋、界面符合要求 |
|    | reversi模式 | 按照规则下棋、界面符合要求 |
|    | gomoku模式  | 按照规则下棋、界面符合要求 |
|    | 其他        | 创建游戏、切换游戏     |
|    |           | 演示模式          |
| 文档 | 内容完整      | 见提交要求         |